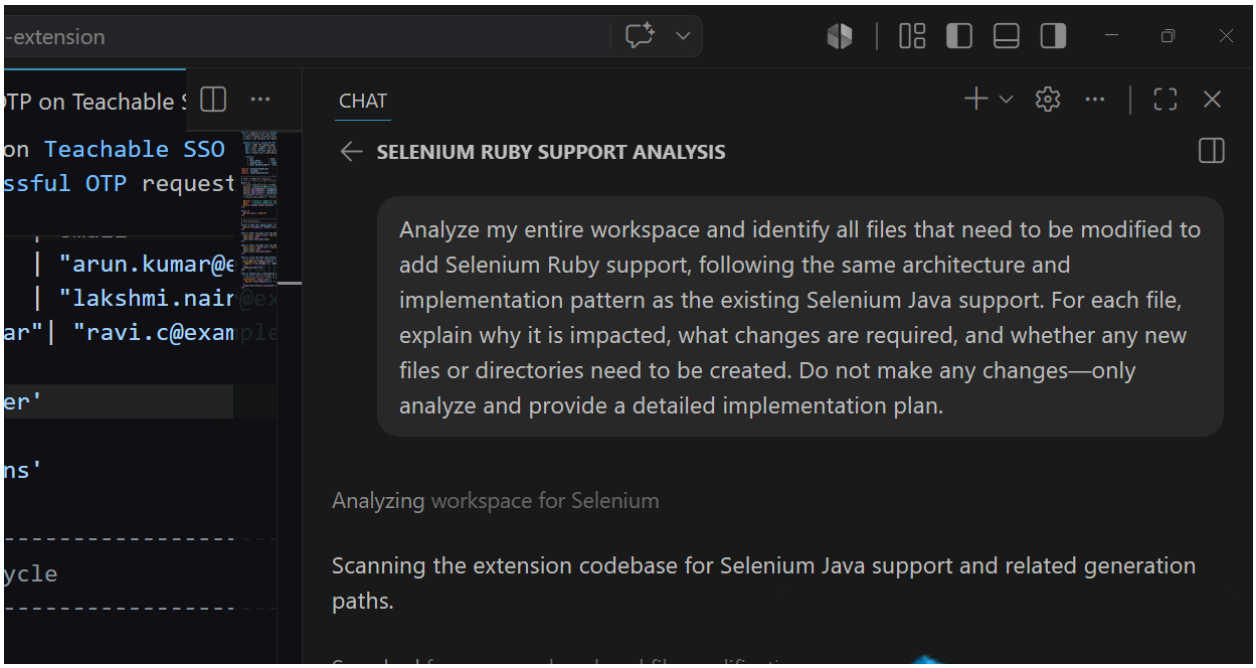
Implementing Selenium Ruby Support in the Code Generator App Through Vibe Coding

Prompts used in vibe coding:

Ask Mode:
Analyze my entire workspace and identify all files that need to be modified to add Selenium Ruby support, following the same architecture and implementation pattern as the existing Selenium Java support. For each file, explain why it is impacted, what changes are required, and whether any new files or directories need to be created. Do not make any changes—only analyze and provide a detailed implementation plan.



Impacts given by Copilot:
Files impacted

1. prompts.js
2. chat.js
3. panel.html

No new files or directories are strictly required for this addition. The current architecture uses prompt templates plus UI selection logic, so Ruby support can be added by extending those existing layers.

Detailed implementation plan

1. prompts.js

Why impacted:

- This file defines the Selenium Java prompt templates and exports the generator key constants.

Required changes:

- Add a new prompt template for Ruby page object generation, e.g. `SELENIUM_RUBY_PAGE_ONLY`.
  - Mirror the Java prompt structure.
  - Instruct generation of only a Selenium Ruby Page Object class.
  - Use Ruby/Selenium idioms and imports such as `require 'selenium-webdriver'`.
  - Keep output format as a single ```` ```ruby ```` code block.
- Add a new prompt template for combined Cucumber + Selenium Ruby step definitions, e.g. `CUCUMBER_WITH_SELENIUM_RUBY_STEPS`.
  - Mirror the existing Java step-definition prompt.
  - Instruct generation of:
    - a Gherkin feature file
    - a Ruby step definition file using Cucumber and Selenium
  - Include Ruby idioms (`Given`, `When`, `Then`, `Before`, `After`, snake_case method names, explicit waits via `Selenium::WebDriver::Wait`).
- Add the new prompt keys to CODE_GENERATOR_TYPES.
  - For example:
    - `SELENIUM_RUBY_PAGE_ONLY`
    - `CUCUMBER_WITH_SELENIUM_RUBY_STEPS`

Why no new file:

- Prompt templates are centralized in prompts.js, so Ruby support fits into the existing prompt file.

---

2. chat.js

Why impacted:

- This file determines which prompt keys to use based on selected language/engine and UI generation mode.
- It also contains the supported language/engine helper methods and the unsupported-combination message.

Required changes:

- Add a Ruby/Selenium detection helper, e.g. isRubySelenium(language, engine).
- Update getPromptKeys(language, engine) to handle Ruby + Selenium:
  - If language === 'ruby' and engine === 'selenium':

        - `PAGE` only => `SELENIUM_RUBY_PAGE_ONLY`
        - `FEATURE + PAGE` => `CUCUMBER_WITH_SELENIUM_RUBY_STEPS`
        - `FEATURE` only => likely CUCUMBER_ONLY (generic feature-only prompt remains valid)
- Update fallback/unsupported-language messaging:
    - Remove or generalize the hard-coded “Only Java/Selenium is currently available” text, since Ruby/Selenium will now be supported.
    - If you want full parity, explicitly include Ruby/Selenium as supported in the message.
- Optionally:
    - If you want the default codeGeneratorType to reflect language selection, adjust the default or generation type logic. But this is not strictly required for support.

Why no new file:

- chat.js already contains the language/engine routing and prompt selection logic.

---

3. panel.html

Why impacted:

- This is the UI that lets the user choose the language binding.

Required changes:

- Add a Ruby option to the `languageBinding` select dropdown.
    - e.g. `<option value="ruby">Ruby</option>`
- No need to change the browser engine dropdown.
- No new UI components are necessary because the feature/page checkbox controls already cover the same generation modes.

Optional note:

- The checkbox group currently uses `name="javaGenerationMode"`, which is a legacy name. It is not required to change for Ruby support, but if you want cleaner architecture you could rename it to something generic like `generationMode` and update chat.js accordingly.

New files/directories

- None required.
- Ruby support can be added entirely within the existing files.

Summary

The full implementation path is:

- prompts.js — add Ruby prompt templates and new generator constants
- chat.js — add Ruby/Selenium detection and map Ruby combos to prompt keys
- panel.html — add Ruby to the language dropdown

That is the complete impact surface in the current workspace.

Agent Mode:
Please analyze the entire workspace and understand how Selenium Java has been implemented throughout the project. Use the existing Selenium Java implementation as the reference architecture and coding pattern.

I want to add support for **Selenium Ruby** with the same level of functionality and integration.

Tasks

Step 1: Update `prompts.js`

Add the following prompt templates by following the existing Selenium Java implementations:

* `SELENIUM_RUBY_PAGE_ONLY`
* `SELENIUM_RUBY_WITH_CUCUMBER`
* `CUCUMBER_WITH_SELENIUM_RUBY_STEPS`

Ensure the prompt format, naming conventions, placeholders, and coding style are consistent with the existing Selenium Java templates.

Step 2: Update `chat.js`

Modify the `getPromptKeys()` method to correctly handle Selenium + Ruby combinations.

The implementation should mirror the existing logic used for Selenium Java while supporting the new Selenium Ruby prompt templates.

Step 3: Update `panel.html`

Add the required UI logic to support **Selenium Ruby**, following the same implementation pattern currently used for Selenium Java.

This includes:

* Displaying the appropriate Selenium Ruby options.
* Mapping the selected options to the correct prompt templates.
* Ensuring the UI behavior is consistent with the existing Selenium Java implementation.

Requirements

* Reuse the existing architecture and coding patterns wherever possible.
* Do not modify or break any existing Selenium Java functionality.
* Keep the implementation consistent with the project's current coding style.
* If additional files need to be updated to fully support Selenium Ruby, identify them and implement the necessary changes.
* Before making any changes, analyze the project structure and dependencies to ensure the new implementation integrates seamlessly with the existing codebase.

Execution Instructions

* **Implement the changes step by step.**
* **Complete only one step at a time.**
* **After completing each step, stop and provide a summary of the changes made.**
* **Do not proceed to the next step until I explicitly approve it.**
* **If you identify any issue, ambiguity, or additional required changes during a step, explain them and wait for my approval before proceeding.**

Copilot implements the changes step by step

🎉 All Three Steps Complete!

Summary of Full Implementation:

Step 1 ✅ — prompts.js

- Added SELENIUM_RUBY_PAGE_ONLY prompt template
- Added CUCUMBER_WITH_SELENIUM_RUBY_STEPS prompt template
- Updated CODE_GENERATOR_TYPES with Ruby keys

Step 2 ✅ — chat.js

- Added isRubySelenium() helper method

- Updated `getPromptKeys()` to route Ruby/Selenium → correct prompt templates
- Updated error message to reflect new supported combinations

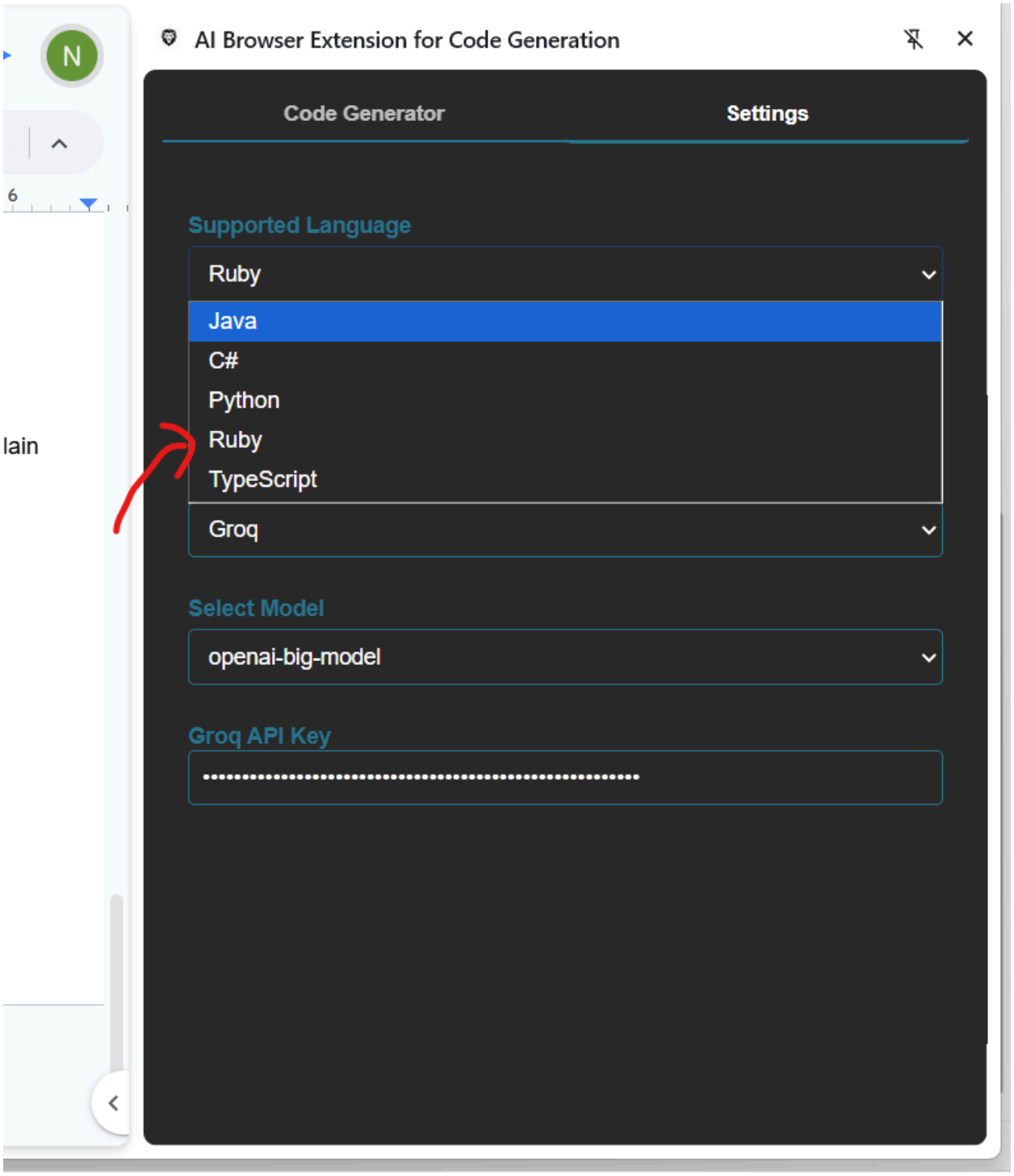
Step 3 ✅ — panel.html

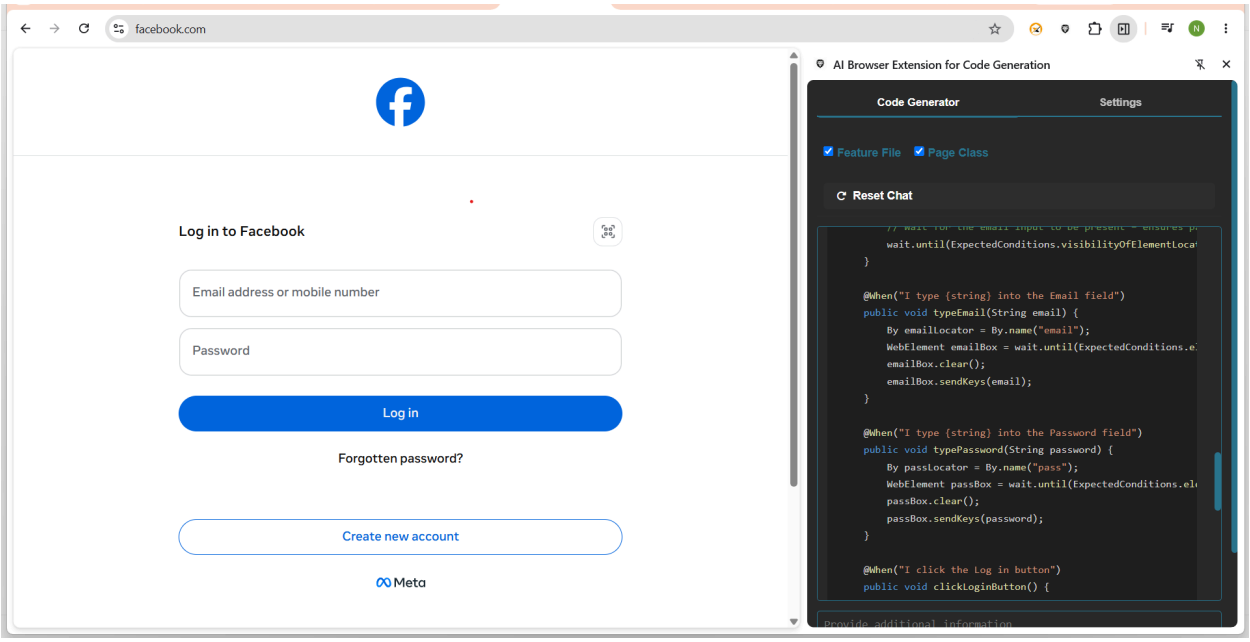
- Added Ruby option to language dropdown

Selenium Ruby Support Flow:

Status: Implementation complete and ready for testing ✓

After the Implementation





The Ruby code + Feature file generated:

```
Feature: Facebook login functionality

  As a QA Engineer
  I want to verify both successful and unsuccessful login attempts on
Facebook
  So that the authentication flow works correctly for users from South
India.

  Background:
    Given I open the Facebook login page

  Scenario Outline: Successful login with valid credentials
    When I type "<email>" into the Email field
    And I type "<password>" into the Password field
    And I click the Log in button
    Then I should be logged in successfully

    Examples: Valid South-India users
      | email                     | password      |
      | "ram.kumar@tcs.com"       | "Ram@12345"   |
      | "sita.patel@infosys.in"   | "Sita#2023"   |
      | "arun.reddy@wipro.co.in"  | "Arun$5678"   |
```

```
      | "lakshmi.n@hcl.com"        | "Lakshmi!9"    |
      | "kavitha.m@techmahindra.in"| "Kavitha%01" |

  Scenario Outline: Unsuccessful login with invalid credentials
    When I type "<email>" into the Email field
    And I type "<password>" into the Password field
    And I click the Log in button
    Then I should see an authentication error message

    Examples: Invalid South-India users
      | email                      | password      |
      | "ram.kumar@tcs.com"        | "wrongPass1"  |
      | "sita.patel@infosys.in"    | "123456"      |
      | "unknown.user@xyz.com"     | "anyPass!"    |
      | "lakshmi.n@hcl.com"        | ""            |
      | "kavitha.m@techmahindra.in"| "short"       |
```

```
package com.facebook.stepdefs;

import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.en.*;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;

import java.time.Duration;

import static org.junit.Assert.*;

public class FacebookLoginSteps {

    private WebDriver driver;
    private WebDriverWait wait;
    private static final String URL = "https://www.facebook.com/";

    @Before
    public void setUp() {
        // Assuming chromedriver is on the system PATH
```

```
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
        driver.manage().window().maximize();
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }

    @Given("I open the Facebook login page")
    public void openLoginPage() {
        driver.get(URL);
        // Wait for the email input to be present - ensures page is loaded

wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("email"))
);
    }

    @When("I type {string} into the Email field")
    public void typeEmail(String email) {
        By emailLocator = By.name("email");
        WebElement emailBox =
wait.until(ExpectedConditions.elementToBeClickable(emailLocator));
        emailBox.clear();
        emailBox.sendKeys(email);
    }

    @When("I type {string} into the Password field")
    public void typePassword(String password) {
        By passLocator = By.name("pass");
        WebElement passBox =
wait.until(ExpectedConditions.elementToBeClickable(passLocator));
        passBox.clear();
        passBox.sendKeys(password);
    }

    @When("I click the Log in button")
```

```
    public void clickLoginButton() {
        // The visible "Log in" text is inside a <span>. Use an XPath that
is stable.
        By loginBtn = By.xpath("//span[text()='Log in' or
normalize-space()='Log in']");
        WebElement btn =
wait.until(ExpectedConditions.elementToBeClickable(loginBtn));
        btn.click();
    }

    @Then("I should be logged in successfully")
    public void verifySuccessfulLogin() {
        // After a successful login Facebook redirects to the home feed.
        // Verify by waiting for the profile navigation element to be
visible.
        By profileNav = By.xpath("//div[@aria-label='Account' or
@aria-label='Profile']");
        WebElement profile =
wait.until(ExpectedConditions.visibilityOfElementLocated(profileNav));
        assertTrue("Profile navigation should be displayed after login",
profile.isDisplayed());

        // Optional: also assert that URL no longer contains "login"
        assertFalse("URL should not contain 'login' after successful
login",
                driver.getCurrentUrl().toLowerCase().contains("login"));
    }

    @Then("I should see an authentication error message")
    public void verifyLoginError() {
        // Facebook shows an error box with role="alert". Locate it.
        By errorBox = By.xpath("//div[@role='alert' or
contains(@class,'_9ay7')]");
        WebElement error =
wait.until(ExpectedConditions.visibilityOfElementLocated(errorBox));
        assertTrue("Error message should be displayed for invalid
credentials", error.isDisplayed());

        // Verify that the error text contains a known phrase (e.g.,
"incorrect")
```

```
        String errText = error.getText().toLowerCase();
        assertTrue("Error text should indicate authentication failure",
                errText.contains("incorrect") ||
errText.contains("wrong"));
    }
}
```